ASTROMESH

OPEN SOURCE · PyPI



Opinión impopular

Dejá de picar tu conocimiento **en pedacitos.**

Libero **astromesh-okf**: tus agentes navegan el conocimiento como una persona, en vez de recuperar fragmentos sueltos.

3 tools | 9 frameworks | MCP | pip install astromesh-okf

Juan Carlos Romero Fulfaro

desliza →

Un fragmento no sabe **si sigue vigente.**

Tampoco quién lo revisó, ni a qué otro documento apunta. Una persona no lee así: abre el índice, sigue los links y mira la fecha.

UN CHUNK	UN CONCEPTO OKF
Un pedazo sin su contexto	El documento entero
Nadie sabe quién lo verificó	trust: human-reviewed
Lo viejo y lo nuevo, mezclados	stale, status y warnings
Links rotos o perdidos	Links que el agente puede seguir

Lee como lo harías **vos.**

```
kb_index("/")
  policies/  products/
kb_find(type="Policy")
  policies/refunds · human-reviewed
kb_read("policies/refunds")
  links → products/plans ✓
          policies/refunds-2024 ✓
kb_read("policies/refunds-2024")
  ⚠ DEPRECATED: do not use for new work
```

Índice → búsqueda → concepto → link. Y cuando llega a la política vieja, **la lee con la advertencia puesta.**

Lo vencido se lee.
Pero avisa.

Leer nunca esconde nada.

Un concepto deprecado o sin verificar se abre igual, con `trust`, `stale`, `status` y `warnings`. El modelo decide sabiendo.

Buscar sí filtra.

`kb_find` saltea lo deprecado, porque un resultado de búsqueda es una recomendación, y una política retirada no debería serlo.

Y puede citar de dónde sacó la respuesta.

Cada concepto tiene un id. La respuesta sale de `policies/refunds`, no de "un fragmento por ahí".

Lo que aprendí **construyéndolo.**

Los modelos se equivocan con las rutas.

Por eso cada link vuelve ya resuelto a un id que el agente pasa tal cual.

Los modelos inventan argumentos.

Vi `limit="2"` y `tags="finance"` en vez de una lista. El loop no se cae.

Una herramienta nunca lanza una excepción.

Cada error vuelve como JSON, y un concepto que no existe trae sugerencias.

El índice va en el system prompt.

No cambia entre preguntas: ahorra una llamada por conversación y el prefijo queda cacheable.

¿Mata a RAG?
No.

NAVEGAR GANA	RAG GANA
Políticas	Miles de documentos
Definiciones de métricas	Sin estructura que seguir
Runbooks	Búsqueda por similitud
Catálogos de datos	Más allá de unos miles de conceptos

Se combinan: un retriever para la cola larga y las tools de OKF para el núcleo curado. Varios bundles en un mismo agente con `prefix`.



ASTROMESH

APACHE-2.0

Nació en Astromesh.
Ahora es de todos.

El lector OKF del runtime, extraído como librería. El core de Astromesh ya lo usa como dependencia.

LangChain	**CrewAI**
Google ADK	**OpenAI Agents**
PydanticAI	**LlamaIndex**
smolagents	**MS Agent Framework**

Y cualquier cliente **MCP**, con **pip install astromesh-okf**. Probado en cada CI contra los 4 bundles de ejemplo de Google: 53 conceptos, ninguno descartado.

Instalalo hoy `github.com/monaccode/astromesh-okf`